colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=jU1D5M9cs2EX

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

Lab Work — Reviewing Python

Student Information

Name: Odumosu Oluwasomidotun Adebowale
Course: Modern Data Analysis
Program: Computational Modeling in Technology and Finance
Lecturer: Professor Prozorov O.A.

This notebook contains my answers to the 14 exercises given for the lab.

1. Enter a three-digit number and find the sum of its digits.

The number is read as an integer. I separate the hundreds, tens and units digits and then add them.

[1]
✓ 7s

```
number = int(input("Enter a three-digit number: "))

if 100 <= number <= 999:
    hundreds = number // 100
    tens = (number // 10) % 10
    units = number % 10
```

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=jU1D5M9cs2EX

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] ✓ 7s

```
    units = number % 10

    digit_sum = hundreds + tens + units
    print("Sum of the digits:", digit_sum)
else:
    print("Please enter a three-digit number.")
```

```
Enter a three-digit number: 583
Sum of the digits: 16
```

Conclusion

The process involves dividing and mod-ing each digit and then adds them together.

2. Compute Formulas:

$$y = -\frac{b + \sqrt{b^2 - 4ac}}{2a}, \quad y = \sin^2(x) + \log_2 |1 + x|. \tag{1}$$

[2] ✓ 1m

```
import math

a = float(input("Enter a: "))
```

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkdClS#scrollTo=jU1D5M9cs2EX

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[2] 1m

```
a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))

discriminant = b**2 - 4*a*c

if discriminant >= 0:
    y1 = (-b + math.sqrt(discriminant)) / (2*a)
    print("Result of first formula =", y1)
else:
    print("No real solution")

x = float(input("Enter x: "))

y2 = math.sin(x)**2 + math.log2(abs(1 + x))

print("Result of second formula =", y2)
```

```
Enter a: 1
Enter b: -3
Enter c: 2
Result of first formula = 2.0
Enter x: 3
Result of second formula = 2.019914856674817
```

Conclusion

The first formula uses the quadratic formula using the discriminant method, while the second formula combines a trigonometric function

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkdClS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

The first formula uses the quadratic formula using the discriminant method, while the second formula combines a trigonometric function with a logarithmic function.

3. "Guess the number" game.

A random number between 1 and 100 was chosen by the computer. The player keeps guessing until the correct number is entered.

[5] 1m

```
import random

number = random.randint(1, 100)

while True:
    try:
        guess = int(input("Guess the number (1-100): "))
    except ValueError:
        print("Invalid input. Please enter a whole number.")
        continue

    if guess == number:
        print("Congratulations! You guessed the correct number.")
        break
    elif guess < number:
        print("The number is higher.")
    else:
        print("The number is lower.")
```

```
Guess the number (1-100): 50
```

Variables Terminal Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkdCIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
Guess the number (1-100): 50
The number is higher.
Guess the number (1-100): 75
The number is lower.
Guess the number (1-100): 63
The number is lower.
Guess the number (1-100): 64
The number is lower.
Guess the number (1-100): 55
Congratulations! You guessed the correct number.
```

Conclusion

The game ends only when the number is correctly identified.

4. Check if a number contains digits 0 or 5.

I convert the number to a string and check whether either character `'0'` or `'5'` occurs.

[6] ✓ 58s

```
number = input("Enter a number: ")

if "0" in number or "5" in number:
    print("The number contains 0 or 5.")
else:
    print("The number does not contain 0 or 5.")
```

Variables Terminal Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkdClS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
Enter a number: 4507
The number contains 0 or 5.
```

Conclusion

The solution checks whether either digit appears in the entered number.

5. Count four-digit numbers containing digit 0.

A four-digit number is from 1000 to 9999. I check every number and count the ones that contain `0`.

```
count = 0

for number in range(1000, 10000):
    if "0" in str(number):
        count += 1

print("Number of four-digit numbers containing 0:", count)
```

```
Number of four-digit numbers containing 0: 2439
```

Conclusion

Variables Terminal Python 3

colab.research.google.com/drive/1_c495J2T5rpIRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

Conclusion

A loop checks every four-digit number and counts those containing at least one zero.

6. Function returning sum and product of three numbers.

[8] ✓ 0s

```
def sum_and_product(a, b, c):
    total = a + b + c
    product = a * b * c
    return total, product


x, y, z = 2, 3, 4

total, product = sum_and_product(x, y, z)

print("Sum:", total)
print("Product:", product)
```

```
Sum: 9
Product: 24
```

Conclusion

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkdCIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

The function returns two values: the sum and the product.

7. Reverse a string and remove first character using slices.

[9] 5m

```
text = input("Enter a string: ")

reversed_text = text[::-1]
without_first = text[1:]

print("Reversed string:", reversed_text)
print("Without first character:", without_first)
```

```
Enter a string: Computational Modeling
Reversed string: gniledoM lanoitatupmoC
Without first character: omputational Modeling
```

Conclusion

I used Python slicing to reverse a string and remove the first character from a string.

8. Replace tabs/newlines with spaces and remove punctuation.

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

8. Replace tabs/newlines with spaces and remove punctuation.

I use `translate()` to remove punctuation and replace tab/newline characters with spaces. Finally, I remove extra spaces.

[10] ✓ 1m

```
import string

text = input("Enter text: ")

text = text.replace("\t", " ").replace("\n", " ")
text = text.translate(str.maketrans("", "", string.punctuation))
text = " ".join(text.split())

print("cleaned text:", text)
```

```
Enter text: Computational Modeling, Modern Data Analysis and Machine Learning
Cleaned text: Computational Modeling Modern Data Analysis and Machine Learning
```

Conclusion

The text becomes cleaner and easier to process.

9. Remove all max elements from a list (new list).

Variables Terminal Python 3

colab.research.google.com/drive/1_c495J2T5rpIRFD4QitPRfboxKTkdCIS#scrollTo=0f1c40a4

9. Remove all max elements from a list (new list).

First I find the maximum value. Then I create a new list containing every element except that maximum value.

[11] ✓ 0s

```
numbers = [4, 8, 2, 8, 5, 8, 1]

maximum = max(numbers)
new_list = [number for number in numbers if number != maximum]

print("Original list:", numbers)
print("Maximum value:", maximum)
print("New list:", new_list)
```

```
Original list: [4, 8, 2, 8, 5, 8, 1]
Maximum value: 8
New list: [4, 2, 5, 1]
```

Conclusion

I find the maximum value in a list and remove all occurrences of it to create a new list.

10. Sort by (1) last digit ascending, (2) descending.

[12]

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

10. Sort by (1) last digit ascending, (2) descending.

```
numbers = [23, 41, 15, 32, 58, 76, 14]

ascending = sorted(numbers, key=lambda number: number % 10)
descending = sorted(numbers, key=lambda number: number % 10, reverse=True)

print("Ascending by last digit:", ascending)
print("Descending by last digit:", descending)
```

```
Ascending by last digit: [41, 32, 23, 14, 15, 76, 58]
Descending by last digit: [58, 76, 15, 14, 23, 32, 41]
```

Conclusion

Sorting is performed using the last digit of each number.

11. Set comprehension for numbers divisible by 3 or 7.

Here I use the range from 1 to 100 as an example. The range can be changed if a different interval is required.

Variables Terminal Python 3

colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

11. Set comprehension for numbers divisible by 3 or 7.

Here I use the range from 1 to 100 as an example. The range can be changed if a different interval is required.

[13] ✓ 0s

```
numbers = {number for number in range(1, 101) if number % 3 == 0 or number % 7 == 0}

print(numbers)
```

```
{3, 6, 7, 9, 12, 14, 15, 18, 21, 24, 27, 28, 30, 33, 35, 36, 39, 42, 45, 48, 49, 51, 54, 56, 57, 60, 63, 66, 69, 70, 72, 75, 77, 78, 81, 84, 87, 90, 91, 93, 96, 98, 99}
```

Conclusion

Set comprehension provides a simple way to create a collection based on conditions.

12. Count characters in a string using a dictionary.

Each character is used as a dictionary key, and its number of occurrences is stored as the value.

[14] ✓ 2m

```
text = input("Enter a string: ")

character_count = {}
```

Variables Terminal

Python 3

colab.research.google.com/drive/1_c495J2T5rpIRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

12. Count characters in a string using a dictionary.

Each character is used as a dictionary key, and its number of occurrences is stored as the value.

[14] 2m

```
text = input("Enter a string: ")

character_count = {}

for character in text:
    character_count[character] = character_count.get(character, 0) + 1

print("Character counts:")
print(character_count)
```

```
Enter a string: Modern Data Analysis
Character counts:
{'M': 1, 'o': 1, 'd': 1, 'e': 1, 'r': 1, 'n': 2, ' ': 2, 'D': 1, 'a': 3, 't': 1, 'A': 1, 'l': 1, 'y': 1, 's': 2, 'i': 1}
```

Conclusion

I count the number of times each character appears in a string using a Python dictionary.

13. Find a word starting with capital letter and length < 5 (regex).

colab.research.google.com/drive/1_c495J2T5rpIRFD4QitPRfboxKTkdCIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

13. Find a word starting with capital letter and length < 5 (regex).

The regular expression looks for a word that starts with an uppercase English letter and has a total length of 1 to 4 letters.

[15]

```
import re

text = input("Enter text: ")

pattern = r"\b[A-Z][A-Za-z]{0,3}\b"
match = re.search(pattern, text)

if match:
    print("First matching word:", match.group())
else:
    print("No matching word was found.")
```

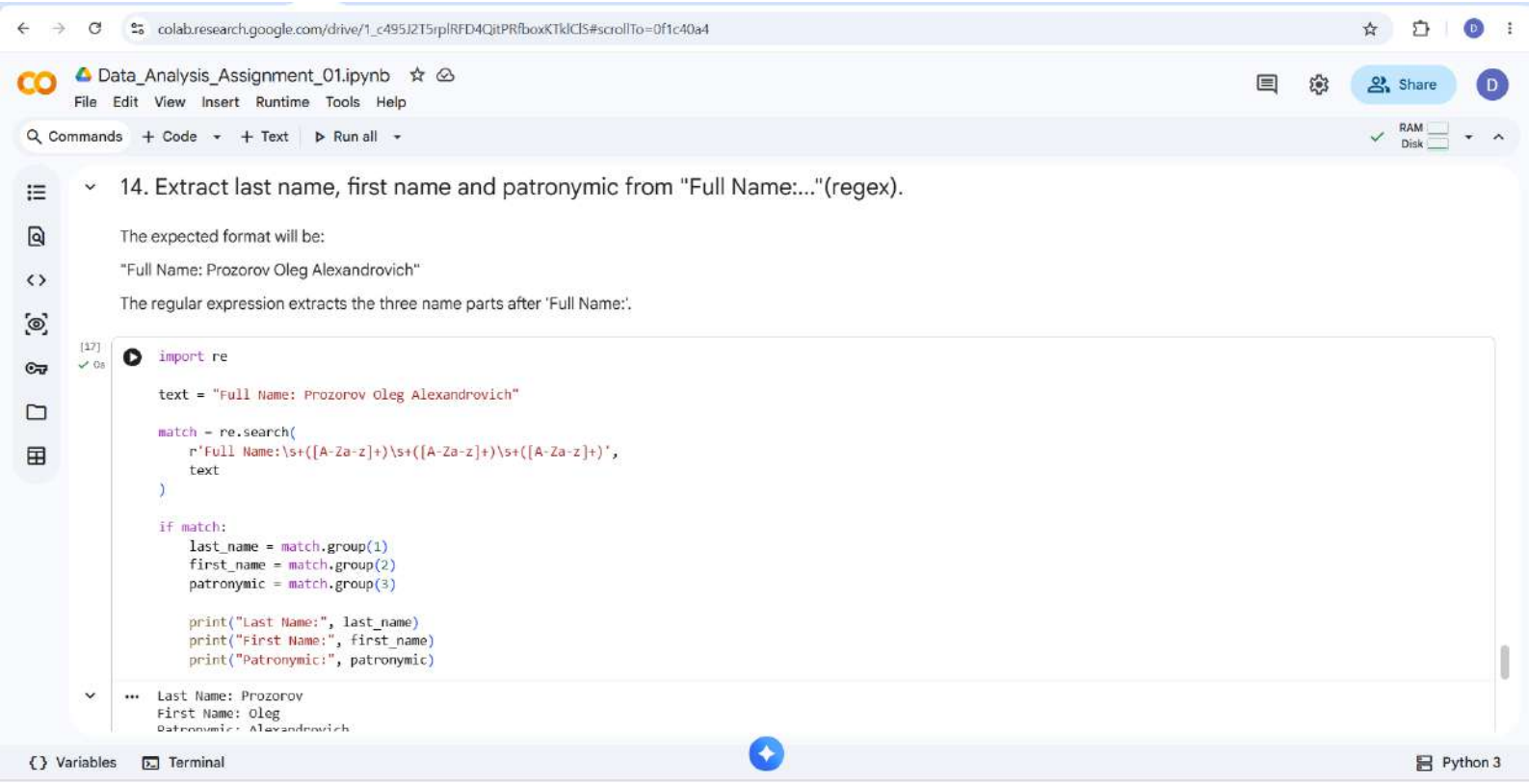
```
Enter text: Data is very good
First matching word: Data
```

Conclusion

I used regular expressions in Python to find a word that matches a specific pattern.

14. Extract last name, first name and patronymic from "Full Name:..."(regex).

Variables Terminal Python 3


colab.research.google.com/drive/1_c495J2T5rplRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4
Data_Analysis_Assignment_01.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
14. Extract last name, first name and patronymic from "Full Name:..."(regex).
The expected format will be:
"Full Name: Prozorov Oleg Alexandrovich"
The regular expression extracts the three name parts after 'Full Name:'.
[17]
import re
text = "Full Name: Prozorov Oleg Alexandrovich"
match = re.search(
r'Full Name:\s+([A-Za-z]+)\s+([A-Za-z]+)\s+([A-Za-z]+)',
text
)
if match:
last_name = match.group(1)
first_name = match.group(2)
patronymic = match.group(3)
print("Last Name:", last_name)
print("First Name:", first_name)
print("Patronymic:", patronymic)
Last Name: Prozorov
First Name: Oleg
Variables
Terminal
Python 3

colab.research.google.com/drive/1_c495J2T5rpIRFD4QitPRfboxKTkICIS#scrollTo=0f1c40a4

Data_Analysis_Assignment_01.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[17] ✓ 0s

```
        print("Last Name:", last_name)
        print("First Name:", first_name)
        print("Patronymic:", patronymic)
```

```
Last Name: Prozorov
First Name: Oleg
Patronymic: Alexandrovich
```

Conclusion

I used regular expressions to validate and separate a full name into its different parts.

Summary

This laboratory work helped me with the opportunity to revise some of the basic concepts in Python programming such as variables, conditions, loops, functions, strings, lists, dictionaries, sets, and regular expressions.

It also increased my confidence in writing basic Python programs and helped in developing good problem-solving skills.

Variables Terminal

Python 3